CHRISTOPHER NEWPORT UNIVERSITY · SCHOOL OF ENGINEERING & COMPUTING

Similarity search in vector databases

kNN, IVF, NSW, HNSW

How a vector database finds the closest vectors without checking every one

Keith Perkins · School of Engineering & Computing

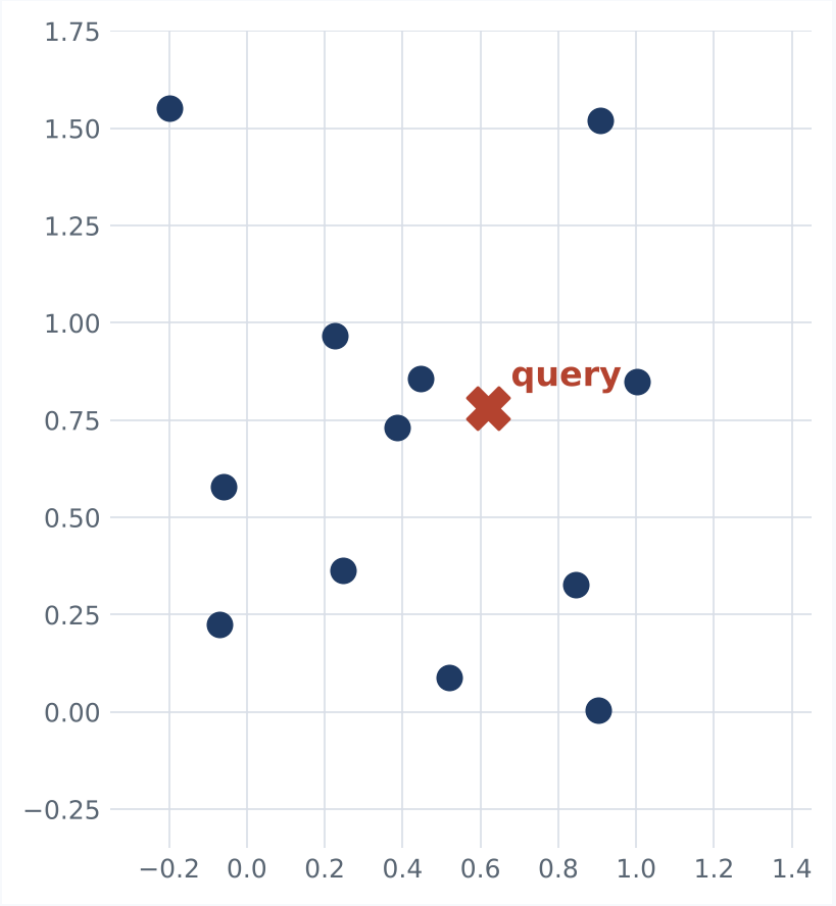
k nearest neighbors (kNN)

Finds the k closest points to a query

- Computes the distance to all n points
- O(n) work per query, so it gets slow at scale
- Always finds the true best matches (exact search)

Which distance? Euclidean (L2) or cosine similarity.

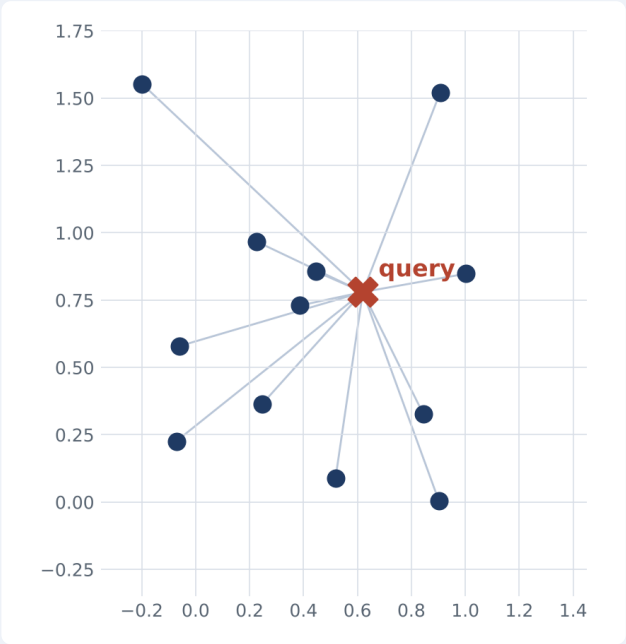
Handy fact: for normalized vectors, $\|a - b\|^2 = 2 - 2 \cdot \cos(a, b)$, so both give the same ranking.



kNN, step by step (k = 3)

STEP 1

Compute the distance to every point



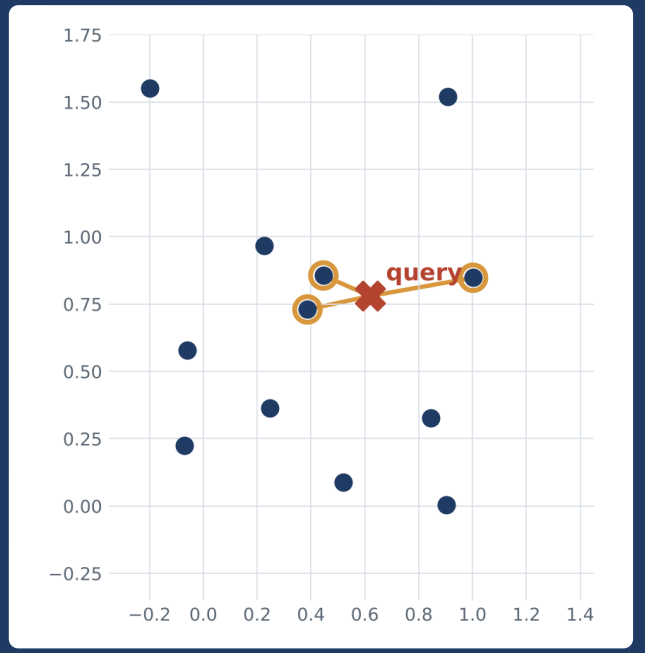
STEP 2

Sort the distances

Unsorted	Sorted
0.89	0.19
0.39	0.24
0.71	0.39
0.70	0.44
0.83	0.51
0.24	0.56
0.19	0.70
…	…

STEP 3

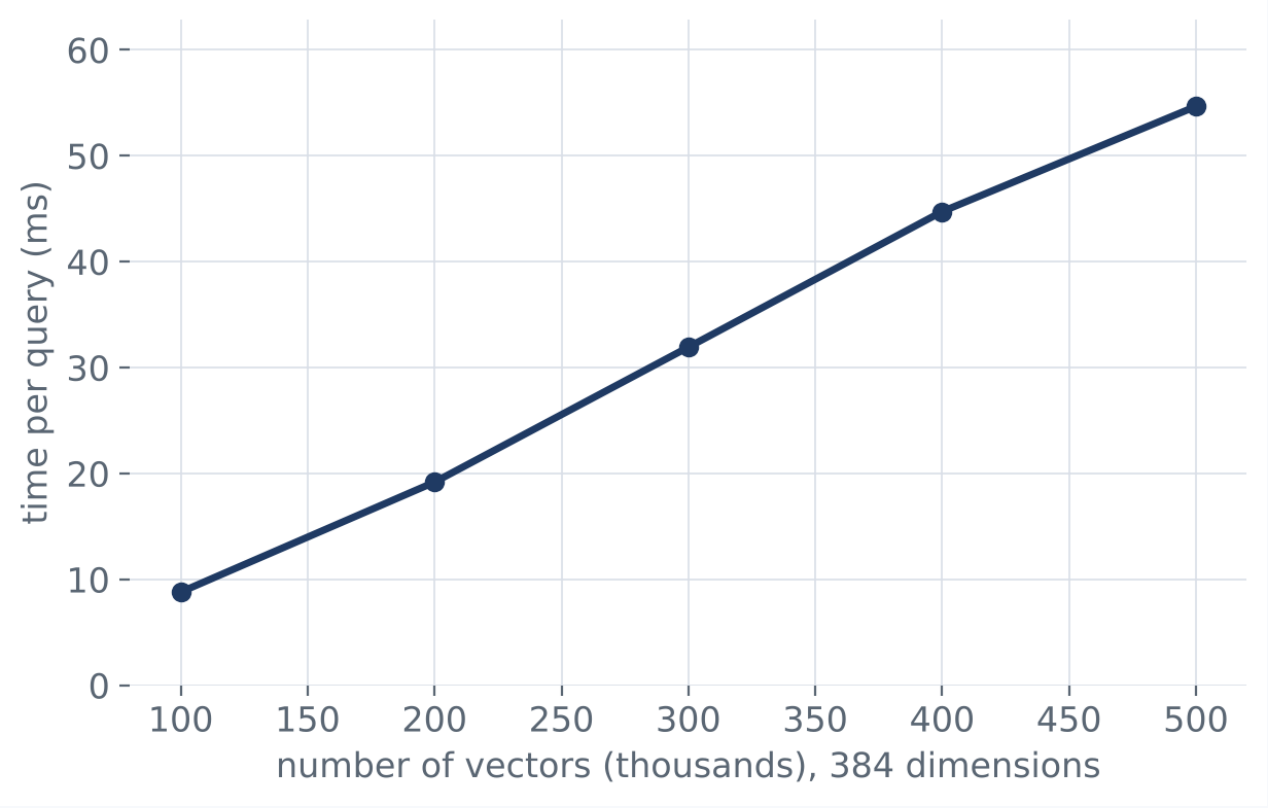
Select the k closest



THE PROBLEM

Why kNN doesn't scale

MEASURED: BRUTE-FORCE SEARCH ON ONE CPU



Per-query time grows linearly with the data: every query is compared to every point.

Extrapolate: ~55 ms for 500K vectors → over 1 second for 10 million, for every single query.

Memory: 10M × 384 numbers × 4 bytes ≈ 15 GB just to hold the vectors.

We need approximate methods.

IVF (inverted file index): the idea

Group the vectors into clusters ahead of time. At query time, search only the few clusters closest to the query.

`nlist` Number of clusters

Chosen at build time. A hyperparameter.

`nprobe` Clusters to search

Chosen at query time. Bigger = more accurate, slower.

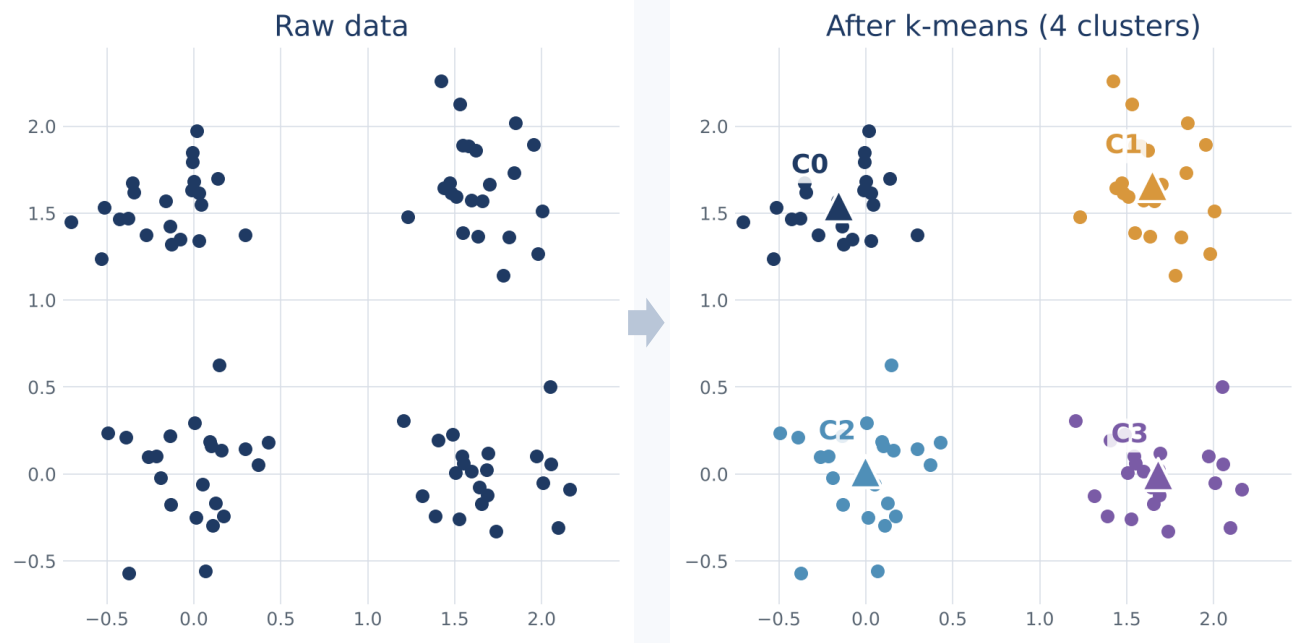
`k` Results to return

Same k as in kNN: how many neighbors you want.

Example: nlist = 1,000 and nprobe = 2: search 2 of 1,000 clusters, about 0.2% of the data.

Rule of thumb (FAISS): nlist ≈ 4√n to 16√n. For 10 million vectors that's roughly 13,000–50,000 clusters.

Build the clusters with k-means (nlist = 4)



C0: list of its points

C1: list of its points

C2: list of its points

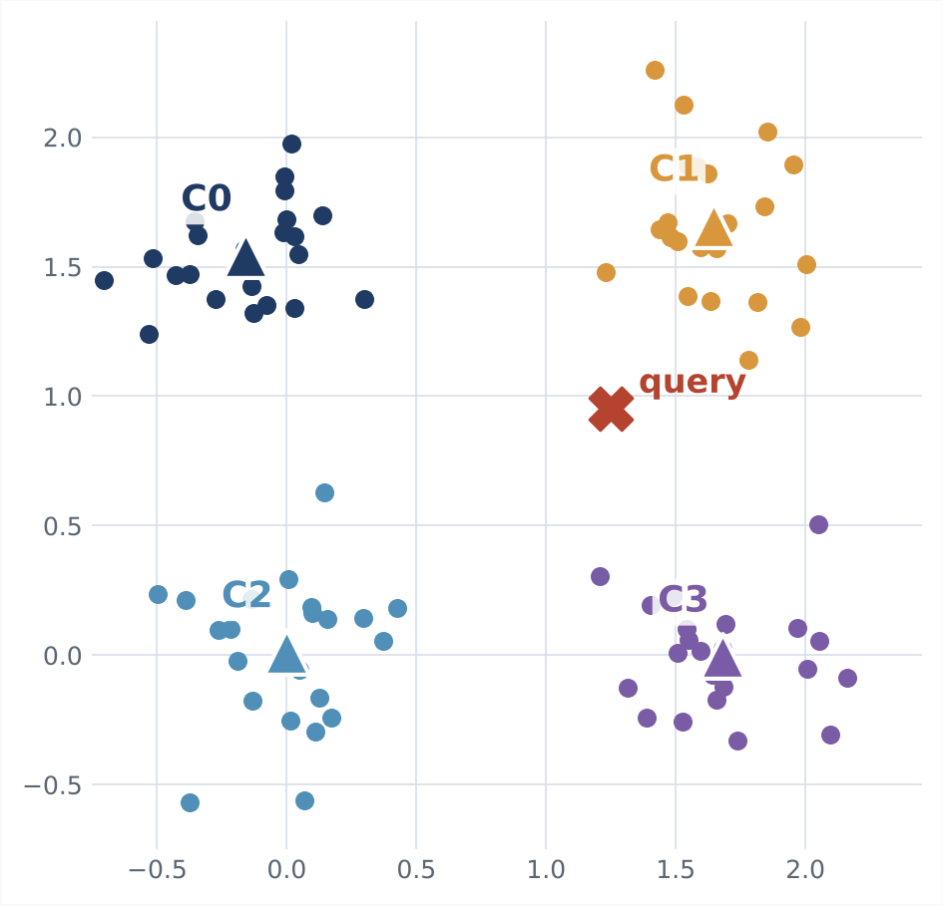
C3: list of its points

K-MEANS: AN ITERATIVE PROCESS

1. Randomly place 4 centroids
2. Assign each point to its closest centroid
3. Move each centroid to the center of its points
4. Repeat from 2 until no point changes cluster

Result: one list of points per cluster. That's the "inverted file."

Find the 3 closest points to the query

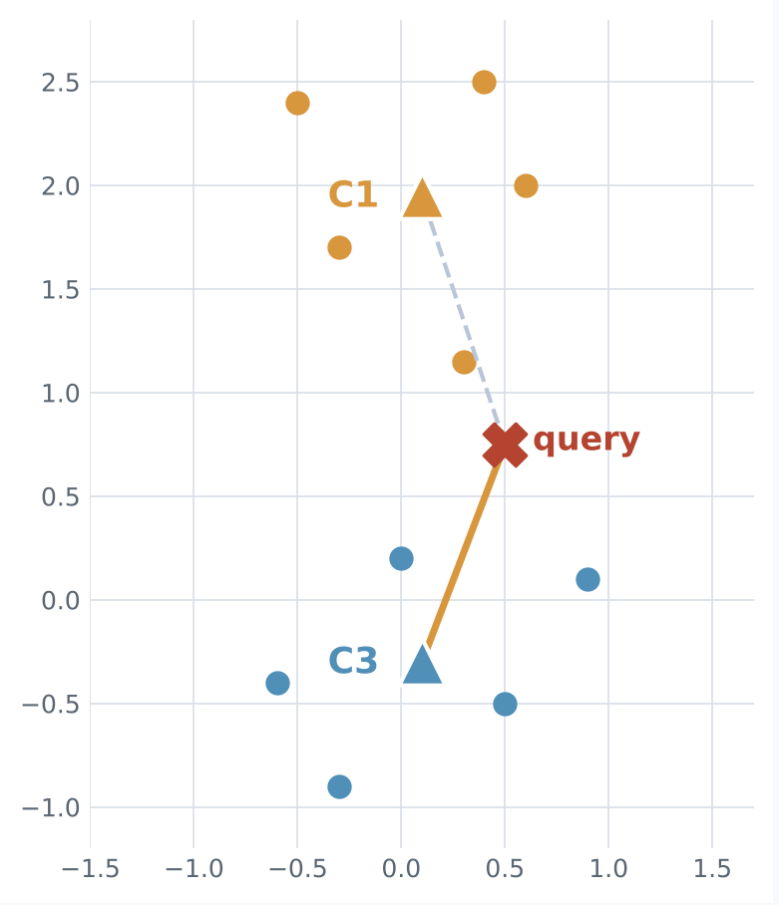


Idea: pick the cluster whose centroid is nearest the query, and search only that cluster?

That's nprobe = 1. Fast, but is it safe? Next slide: a case where it goes wrong.

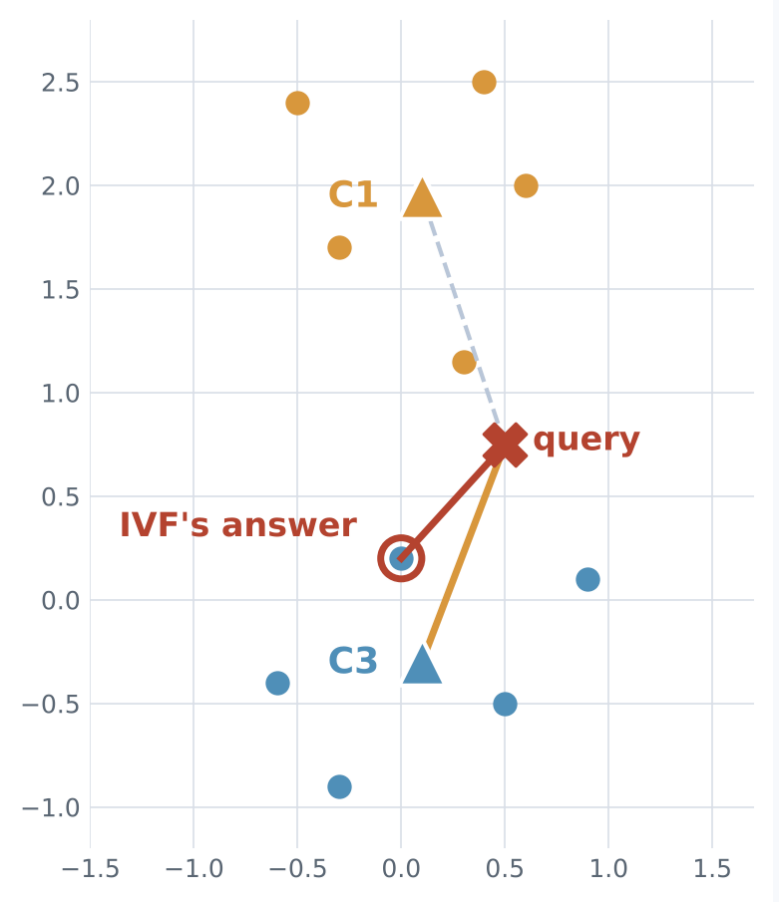
Searching one cluster can miss the answer

1. **The query is closer to C3's centroid than to C1's**
2. So nprobe = 1 searches only C3, and returns C3's closest point
3. But the true nearest point is in C1, right on the cluster's edge



Searching one cluster can miss the answer

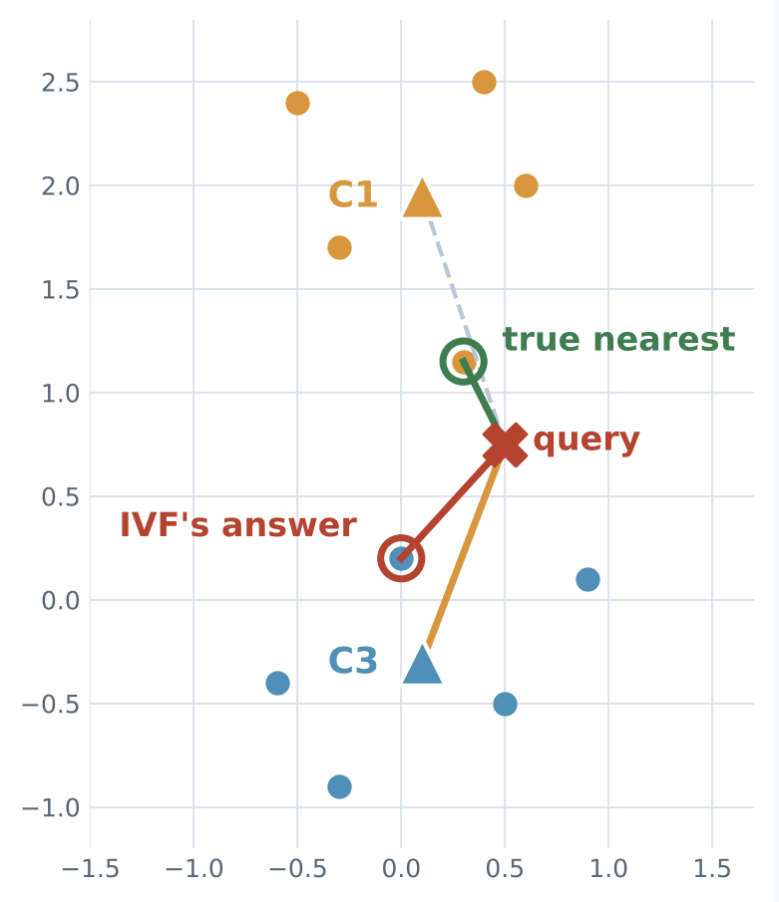
1. **The query is closer to C3's centroid than to C1's**
2. **So nprobe = 1 searches only C3, and returns C3's closest point**
3. But the true nearest point is in C1, right on the cluster's edge



Searching one cluster can miss the answer

1. The query is closer to C3's centroid than to C1's
2. So nprobe = 1 searches only C3, and returns C3's closest point
3. But the true nearest point is in C1, right on the cluster's edge

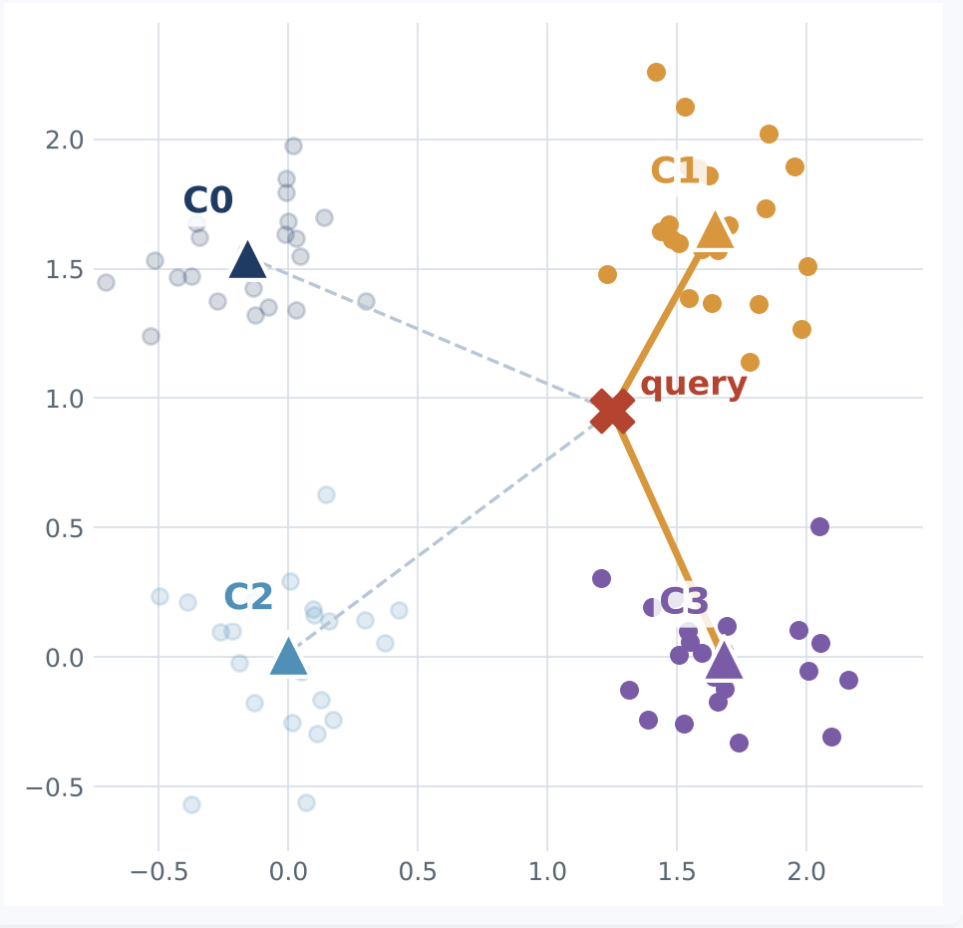
Partial fix: search more than one cluster (nprobe > 1)



Find the 3 closest points, with nprobe = 2

1. **Find the 2 closest centroids (nprobe = 2)**
2. Compute the distance to every point in those 2 clusters
3. Sort the distances and select the closest 3

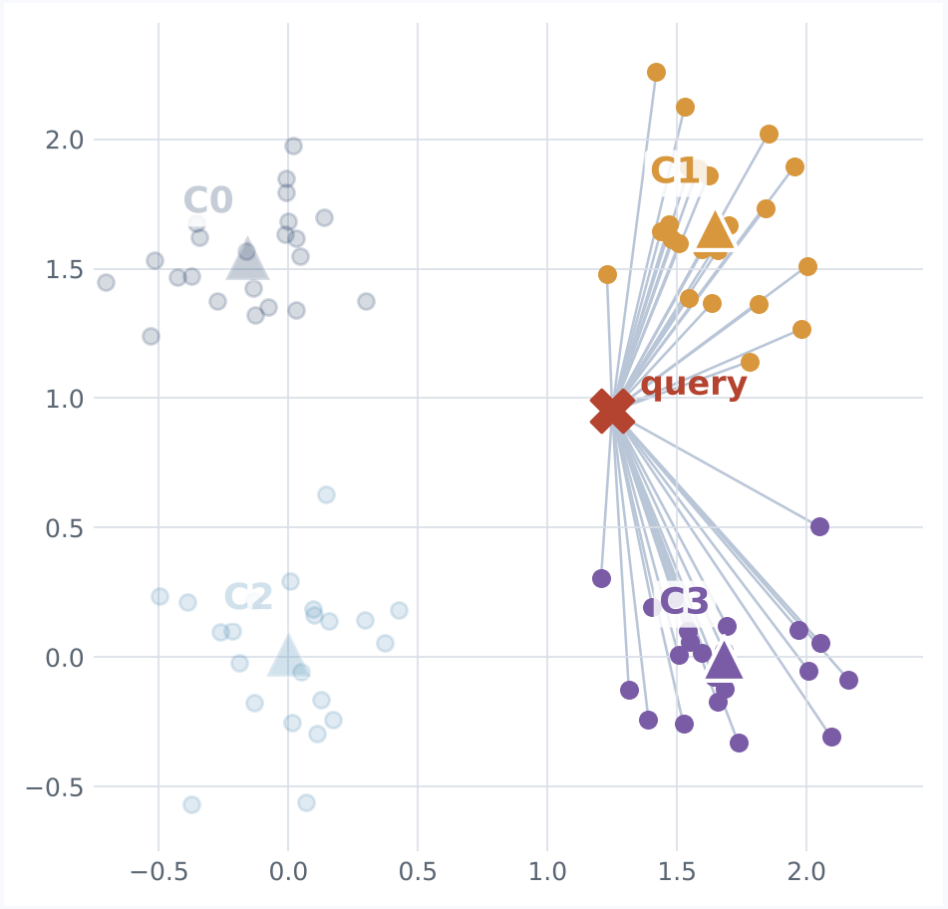
C1 and C3 are the closest centroids. Use their lists to get the points to compare.



Find the 3 closest points, with nprobe = 2

1. **Find the 2 closest centroids (nprobe = 2)**
2. **Compute the distance to every point in those 2 clusters**
3. Sort the distances and select the closest 3

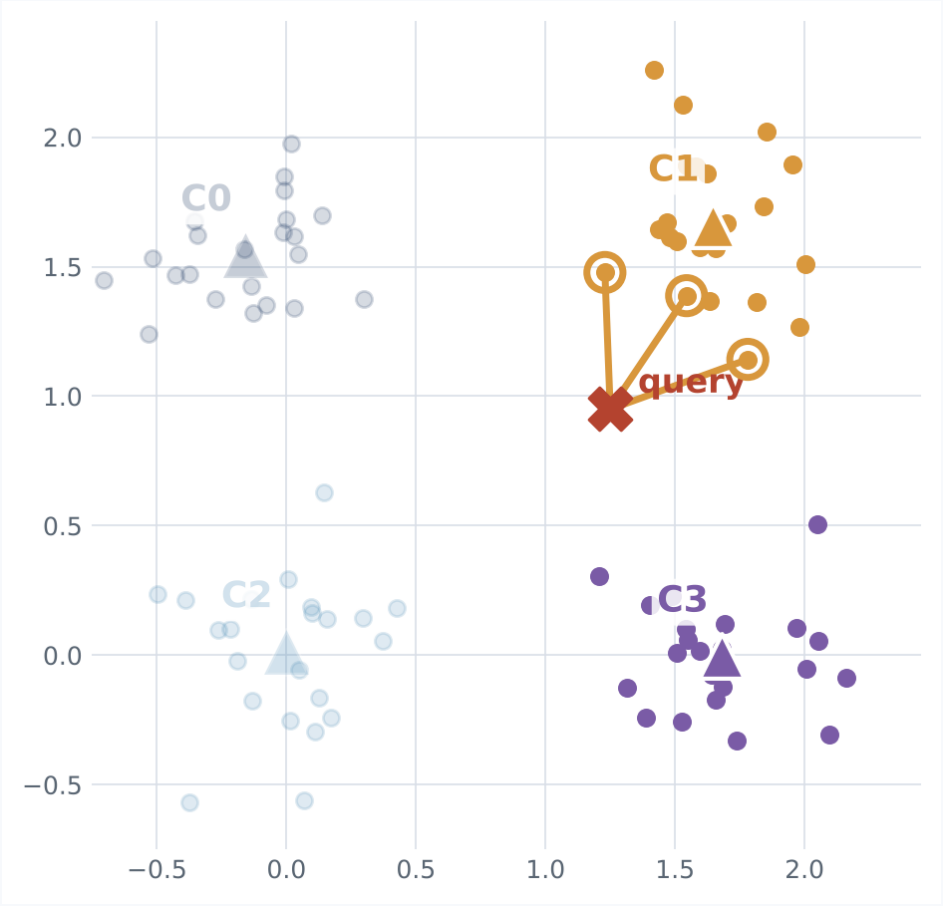
44 of 88 points checked: half the work of kNN here, and far less with real numbers of clusters.



Find the 3 closest points, with nprobe = 2

1. Find the 2 closest centroids (nprobe = 2)
2. Compute the distance to every point in those 2 clusters
3. Sort the distances and select the closest 3

Here IVF found the same 3 points exact kNN would.



How much work does IVF save?

Count distance computations. Assume clusters are equal-sized; IVF compares the query to every centroid, then to every point in the nprobe clusters.

PROBLEM 1

1 million vectors
nlist = 1,000
nprobe = 10

Cluster size=1,000,000/1000=1000

Centroids: 1,000

Points: 10 clusters × 1,000 = 10,000

Total ≈ 11,000 vs 1,000,000 → about 90× less work

PROBLEM 2

10 million vectors
nlist = 4,000
nprobe = 16

Cluster size=10,000,000/4000=2500

Centroids: 4,000

Points: 16 × 2,500 = 40,000

Total ≈ 44,000 vs 10,000,000 → about 230× less work

PROBLEM 3

Problem 1, but nprobe = 1,000

Centroids: 1,000

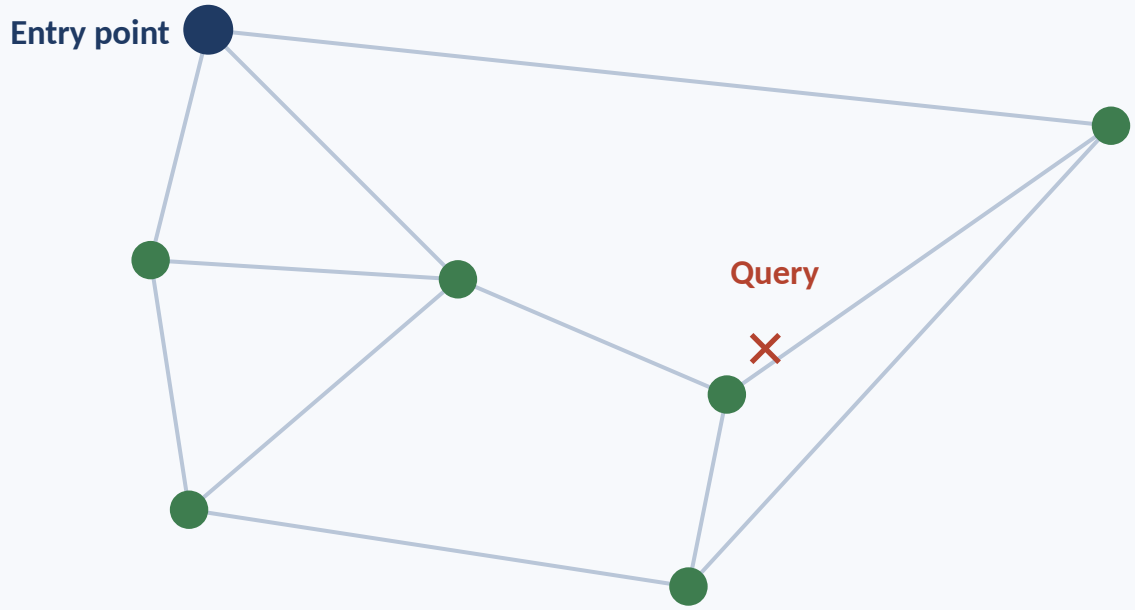
Points: 1000x100 (all 1,000,000)

Total ≈ 1,001,000 → slower than plain kNN! nprobe = nlist means searching everything.

Navigable small worlds (NSW): the idea

- Build a connected graph of all the points
- Connect each point to its nearest neighbors. Connected points are "friends."
- Limit how many friends a point can have
- Each point keeps a list of its friends

Why "small world"? Points added early get some long links. A few long links plus many short ones let you reach any point in a few hops, like "six degrees of separation."

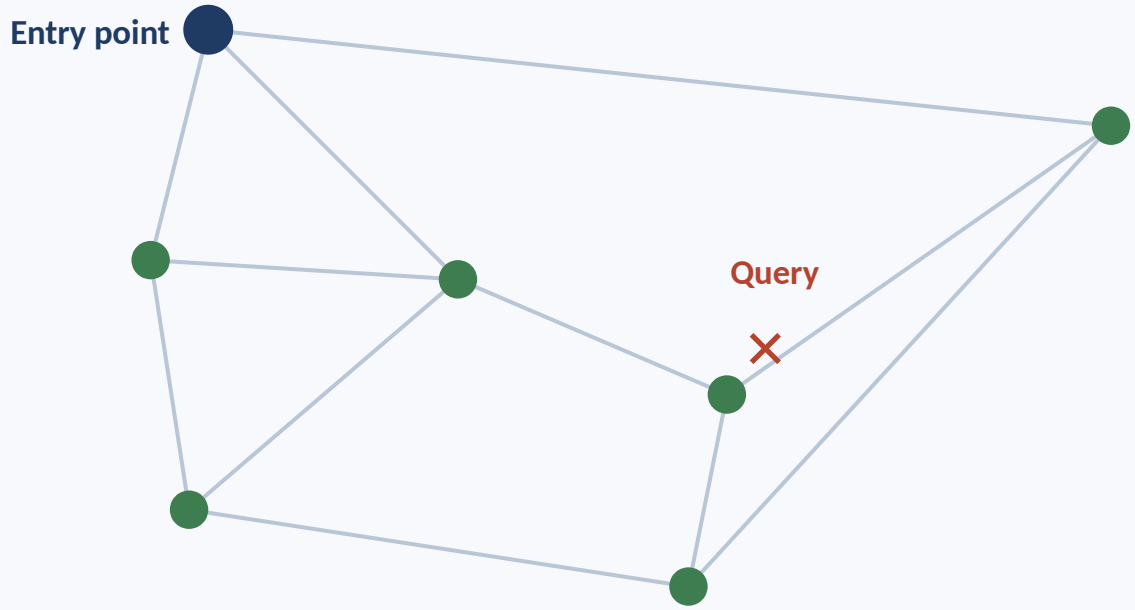


Greedy search through the graph

GREEDY SEARCH

1. **Start at a pre-defined entry point**
2. Greedy step: move to the friend closest to the query
3. If no friend is closer than the current node, stop (a local minimum)
4. Otherwise go to step 2

Start at the entry point.

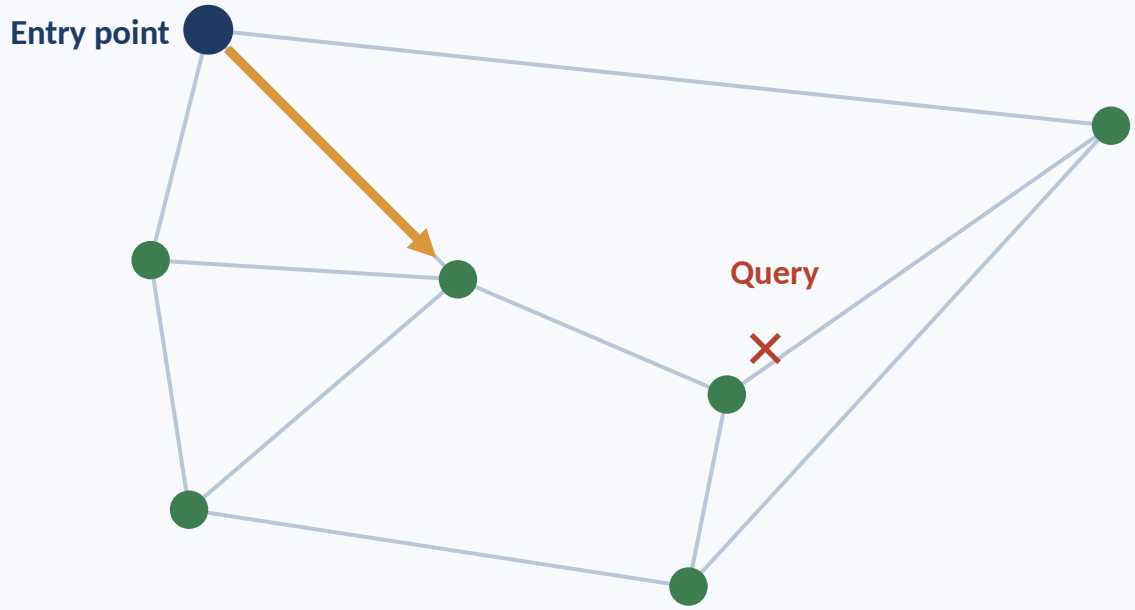


Greedy search through the graph

GREEDY SEARCH

1. Start at a pre-defined entry point
2. **Greedy step: move to the friend closest to the query**
3. If no friend is closer than the current node, stop (a local minimum)
4. Otherwise go to step 2

Of the entry point's friends, this one is closest to the query.

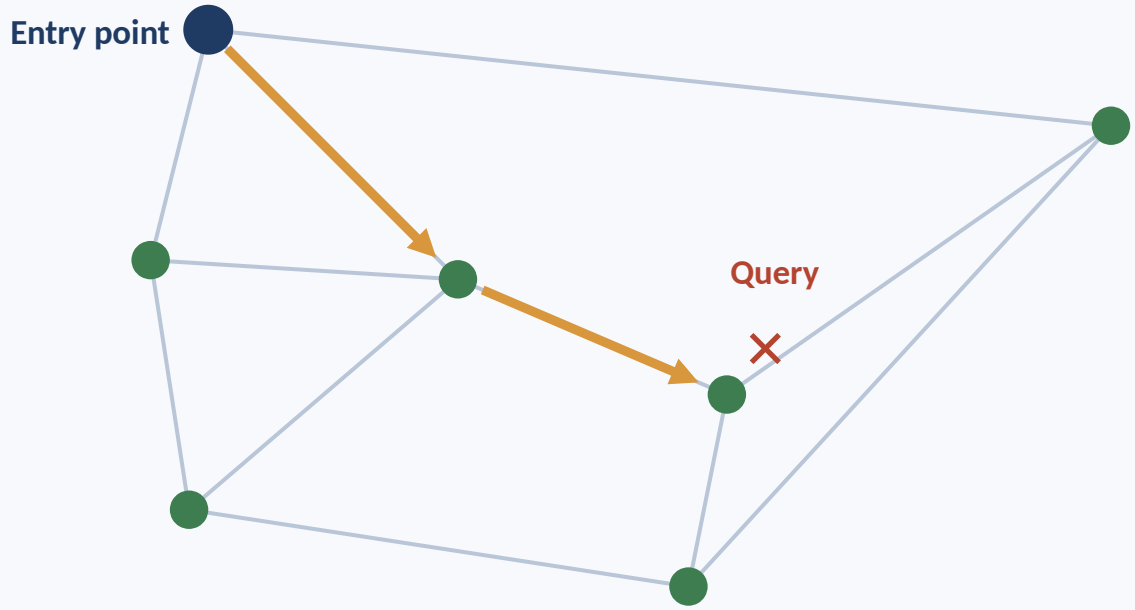


Greedy search through the graph

GREEDY SEARCH

1. Start at a pre-defined entry point
2. **Greedy step: move to the friend closest to the query**
3. If no friend is closer than the current node, stop (a local minimum)
4. **Otherwise go to step 2**

Repeat: hop to the friend closest to the query.

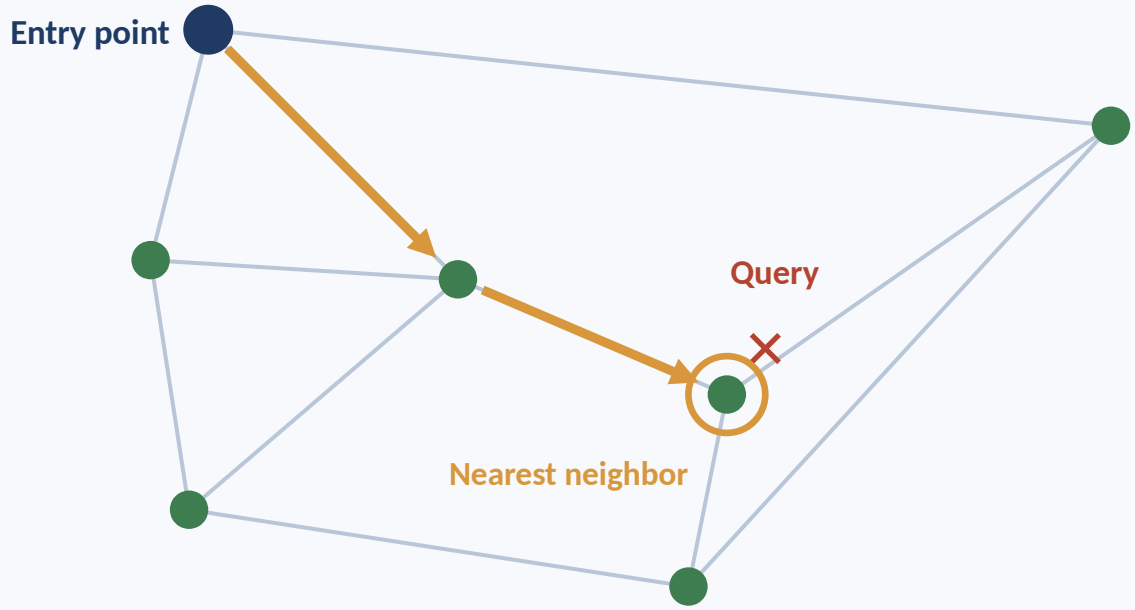


Greedy search through the graph

GREEDY SEARCH

1. Start at a pre-defined entry point
2. Greedy step: move to the friend closest to the query
3. **If no friend is closer than the current node, stop (a local minimum)**
4. Otherwise go to step 2

No friend is closer. Stop: nearest neighbor found.

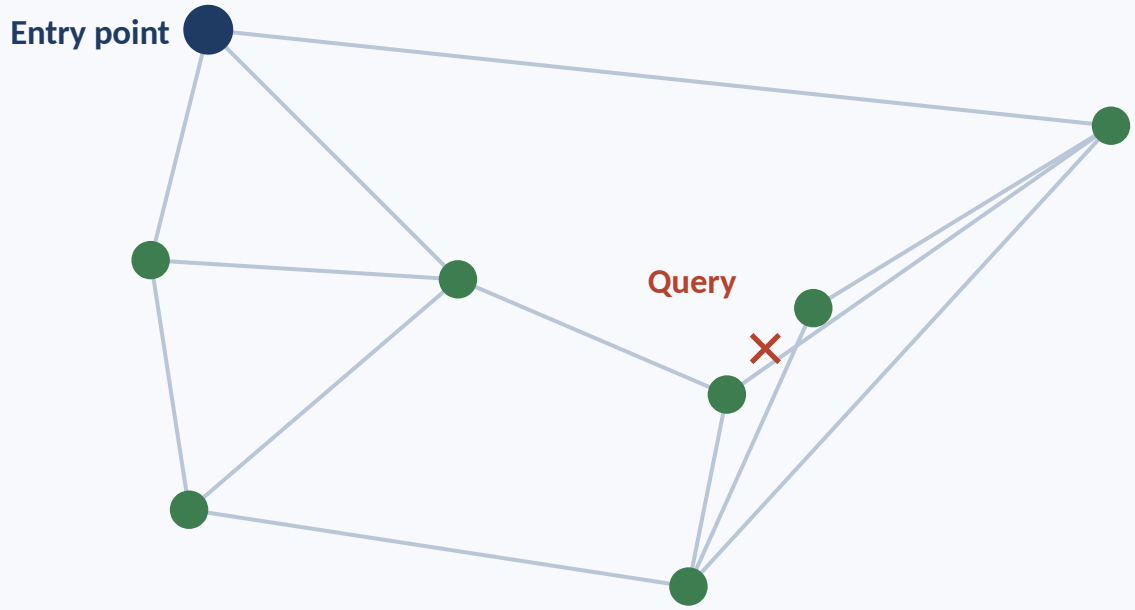


Greedy search can miss the true nearest neighbor

GREEDY SEARCH

1. Start at a pre-defined entry point
2. Greedy step: move to the friend closest to the query
3. If no friend is closer than the current node, stop (a local minimum)
4. Otherwise go to step 2

Add one more point, very close to the query.

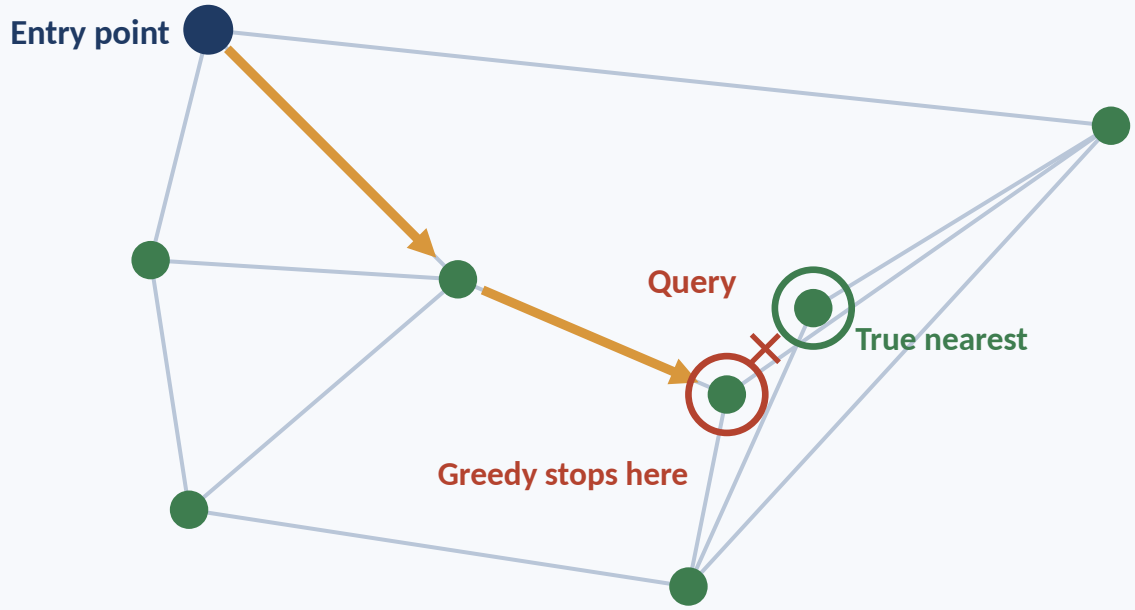


Greedy search can miss the true nearest neighbor

GREEDY SEARCH

1. Start at a pre-defined entry point
2. Greedy step: move to the friend closest to the query
3. If no friend is closer than the current node, stop (a local minimum)
4. Otherwise go to step 2

Greedy never reaches the new, closest point. NSW is an approximation.



Hierarchical navigable small worlds (HNSW)



An evolution of NSW: a stack of NSW graphs.

- Top layers: few nodes, long jumps
- Bottom layer (0): every node, short hops
- Search starts at the top and works its way down

Like an express train for the long distance, then a local train for the last few stops.

Building the layers: two ideas

M

The maximum number of friends (links) a node has in each layer

When a node is added to a layer, it is linked to its M nearest nodes already in that layer.

You choose M. Typical: 16 to 48.

Moving up: a weighted coin flip

Every node starts in layer 0. Flip a coin that says **"up" with probability 1/M**. Up: climb one layer and flip again. Not up: stop. That layer is the node's top layer.

```
level = 0
while random.random() < 1/M: level += 1
```

ADDING ONE NODE

1. **Flip coins to pick its top layer**
2. **Put it in its top layer and every layer below**
3. **In each of those layers, link it to its M nearest nodes**
4. **If it's in a new highest layer, it becomes the entry point**

Result: layer 0 holds every node, and each layer above holds about 1/M of the layer below it.

Example: M = 4, so the chance of moving up is 1/4

ADDING ONE NODE

Flip	Random number	< 0.25?	Now in layer
1	0.18	Yes: up	1
2	0.07	Yes: up	2
3	0.61	No: stop	2

Top layer = 2. The node goes in layers 2, 1 and 0.

In each of those 3 layers it gets linked to its 4 nearest nodes, so it has up to 4 friends per layer.

ALL 1,000 NODES, M = 4

Layer	Chance a node gets here	≈ nodes
Layer 4	$(1/4)^4 = 0.4\%$	4
Layer 3	$(1/4)^3 \approx 1.6\%$	16
Layer 2	$(1/4)^2 \approx 6\%$	63
Layer 1	1/4 = 25%	250
Layer 0	100%	1,000

Each layer ≈ 1/4 of the one below. With the more typical M = 16, it's 1/16 ≈ 6%.

Searching HNSW: top down



1. Start at the entry point in the top layer
2. Greedily move to the friend nearest the query until none is closer
3. Drop down one layer, starting from the same node
4. Repeat from 2 until you finish layer 0

On layer 0, keep efSearch candidates, not just one.

HNSW knobs: speed vs. accuracy

M

BUILD TIME

Maximum friends per node. More links = better recall, more memory. Typical: 16–48.

efConstruction

BUILD TIME

Candidates considered while linking each new node. Higher = better graph, slower build.

efSearch

QUERY TIME

Candidates kept while searching. Higher = better recall, slower queries. Can change anytime.

How do we measure "accuracy"? Recall@k. Run exact kNN and the approximate search on the same query. Recall@k = the fraction of the true top-k that the approximate search found.

Example: exact top-10 vs HNSW top-10 share 9 results → recall@10 = 0.9. Raising M or efSearch pushes recall toward 1.0, at the cost of memory or latency.

BIG PICTURE

Quick comparison

	kNN (exact)	IVF	HNSW
Finds the true nearest?	Always	Usually (approximate)	Usually (approximate)
Query time	Slowest: O(n)	Low; tune with nprobe	Very low; tune with efSearch
Build time	None	Moderate (k-means)	High (graph building)
Memory	Vectors only	Vectors + centroids	Vectors + graph links (most)
In practice	FAISS IndexFlatL2; fine for small data	FAISS IndexIVFFlat, pgvector, Milvus	Chroma (default), FAISS, pgvector, Milvus

efSearch: how many candidates HNSW keeps while searching. Higher = searches more nodes = higher recall, slower.

TRY IT

All three in FAISS

```
import faiss, numpy as np
X = np.random.rand(100_000, 384).astype("float32")    # the vectors
q = np.random.rand(1, 384).astype("float32")          # a query

# Exact kNN: no training, checks everything
flat = faiss.IndexFlatL2(384)

# IVF: k-means into nlist clusters, search nprobe of them
ivf = faiss.IndexIVFFlat(faiss.IndexFlatL2(384), 384, 1000)
ivf.train(X); ivf.nprobe = 8

# HNSW: M = 32 friends per node
hnsw = faiss.IndexHNSWFlat(384, 32)
hnsw.hnsw.efSearch = 64

for index in (flat, ivf, hnsw):
    index.add(X)
    distances, ids = index.search(q, 5)     # top-5
```

Same API for all three

add() the vectors, search() a query. Only the index type and its knob change.

Measure it

Time each search, and compare ids with the flat index to get recall@5.

nlist = 1,000

Best ≈ √(nprobe·n) = √(8 × 100,000) ≈ 894. Round up to 1,000.

SUMMARY

Key takeaways

kNN

Exact, but checks every point: too slow for large n.

IVF

Cluster first, then search only nprobe clusters. Misses near cluster edges.

NSW

A connected graph searched greedily, hop by hop. Can stop early.

HNSW

Layered NSW: long jumps on top, fine search at the bottom. The usual default.

Approximate search trades a little recall for a huge speedup. Knobs like nprobe and efSearch let you choose where on that trade-off to sit.